Homework – Welford's Recurrence and Sampling Mean/Variance

Student: Francesca Papadia | Course: Cybersecurity | ID: 2331411 | Date: 03/11/2026

Ex. 1 – Welford's recurrence formulas

Notation

For observations $x_1, x_2, \ldots$ let $M_k = (x_1 + \ldots + x_k)/k$ be the mean of the first k values and $SS_k = \Sigma_{i=1}^{k} (x_i - M_k)^2$ the sum of squares of deviations from the mean. The sample variance is $s^2_k = SS_k/(k - 1)$ for $k \geq 2$. We set $M_0 = 0$ and $SS_0 = 0$, and define the innovation $\delta = x_k - M_{k-1}$.

Knuth's recurrence for the mean

Since $k \cdot M_k = (k - 1)M_{k-1} + x_k$, dividing by k gives $M_k = M_{k-1} + (x_k - M_{k-1})/k = M_{k-1} + \delta/k$. The mean is updated by a correction proportional to how far the new value is from the old mean.

Welford's recurrence for the sum of squares

We want SS_k in terms of SS_{k-1}. From Knuth, $M_k = M_{k-1} + \delta/k$. Split SS_k into the first $k - 1$ terms and the last term.

1. For $i \leq k - 1$: $x_i - M_k = (x_i - M_{k-1}) - \delta/k$. Squaring and summing over $i = 1, \ldots, k - 1$: $\Sigma(x_i - M_k)^2 = SS_{k-1} - 2(\delta/k)\Sigma(x_i - M_{k-1}) + (k - 1)\delta^2/k^2$. The middle sum is 0 (deviations from the mean sum to zero), so this equals $SS_{k-1} + (k - 1)\delta^2/k^2$.
2. For the last term: $x_k - M_k = \delta - \delta/k = \delta(k - 1)/k$, so $(x_k - M_k)^2 = (k - 1)^2\delta^2/k^2$.
3. Adding: $SS_k = SS_{k-1} + (k - 1)\delta^2/k^2 \cdot (1 + k - 1) = SS_{k-1} + (k - 1)\delta^2/k$.

Since $x_k - M_k = \delta(k - 1)/k$, we have $(k - 1)\delta^2/k = \delta \cdot (x_k - M_k)$. Therefore:

- **$M_k = M_{k-1} + (x_k - M_{k-1})/k$**
- **$SS_k = SS_{k-1} + (x_k - M_{k-1})(x_k - M_k)$**
- **$s^2_k = SS_k/(k - 1)$** (or SS_k/k for the population-style variance)

The two forms of the SS update are equivalent; the second one avoids the product $k - 1$ and uses the old and new means.

Code (core logic: 3 lines)

```
for (let k = 1, M = 0, SS = 0; k <= n; k++) {
  const d = x[k-1] - M;      // innovation
  M  += d / k;               // Knuth
  SS += d * (x[k-1] - M);    // Welford
}                            // sample variance = SS / (n-1)
```

It needs one pass, O(1) memory, and is numerically stable: the naive formula ($\Sigma x^2 - n \cdot M^2$) subtracts two large, nearly equal numbers and can lose all precision. As a check, on 1000 random scores Welford and the classic two-pass formula agree to about 10^{-11}.

Ex. 2 – Sampling mean and variance

Setup

Each penetration attempt gets a score in {0, 1, …, 10}. The chosen distribution is **Binomial(10, 0.4)** (the number of successes in 10 independent tests, each with probability 0.4), with theoretical mean $\mu = 10 \cdot 0.4 = 4$ and variance $\sigma^2 = 10 \cdot 0.4 \cdot 0.6 = 2.4$ (fourth central moment $\mu_4 = 16.224$). The program also offers the discrete uniform distribution on {0..10}, with $\mu = 5$ and $\sigma^2 = 10$.

There are m = 2000 hackers; each attacks n systems and gets a trajectory of n scores. For each hacker, Welford's formulas give the sample mean M_n and the sample variance $s^2 = SS_n/(n-1)$. We then average M_n and s^2 across the hackers and compare with μ and σ^2. The experiment is repeated for n = 2, 5, 10, 50, 100, 1000.

Results (Binomial(10, 0.4); empirical, theory in brackets)

n	Avg sample mean ($\mu = 4$)	Avg s^2 ($\sigma^2 = 2.4$)	Avg SS/n	Var of sample means (σ^2/n)	Var of s^2
2	4.008	2.451	1.225 (1.2)	1.201 (1.2)	12.50 (10.99)
5	3.992	2.420	1.936 (1.92)	0.490 (0.48)	2.764 (2.669)
10	3.976	2.400	2.160 (2.16)	0.2316 (0.24)	1.185 (1.174)
50	3.988	2.408	2.360 (2.352)	0.0470 (0.048)	0.2152 (0.2140)
100	3.995	2.393	2.369 (2.376)	0.0239 (0.024)	0.1031 (0.1058)
1000	3.999	2.397	2.394 (2.398)	0.00243 (0.0024)	0.01027 (0.01048)

The theoretical value of $\mathrm{Var}(s^2)$ is $[\mu_4 - (n-3)\sigma^4/(n-1)]/n$. With the uniform distribution ($\mu = 5$, $\sigma^2 = 10$) the picture is the same: the average sample mean is between 4.99 and 5.05 for every n, the average s^2 between 9.81 and 10.05, and SS/n equals 4.90 at n = 2 (theory 5) and 9.99 at n = 1000.

Conclusions

1. The sample mean is unbiased. Averaged over hackers, the sample mean is very close to μ for every n, even n = 2. In theory $E[M_n] = \mu$ for every n.

2. The sample variance with n − 1 is unbiased; dividing by n is not. The average of s^2 is close to σ^2 for every n, while SS/n is systematically too small, by the factor (n − 1)/n (1.225 against 2.4 at n = 2, about half). Reason: deviations are measured from the sample mean M_n, which is itself fitted to the data and makes the points look closer together than they are. Formally $SS_n = \Sigma(x_i - \mu)^2 - n(M_n - \mu)^2$, so $E[SS_n]$ =

$n\sigma^2 - n\cdot(\sigma^2/n) = (n-1)\sigma^2$. This is why we divide by n − 1 (Bessel's correction). The bias of SS/n vanishes as n grows (2.394 against 2.4 at n = 1000).

3. Why this is useful in inferential statistics. We almost never know μ and σ^2; we observe a sample. Sample mean and sample variance are the standard unbiased estimators of them, and the experiment shows what "unbiased" means: across many repetitions, the estimates are centered on the truth. Welford's recurrence lets us compute them online, in a single pass over data that arrives one at a time (as the scores of a hacker do). They are the ingredients of standard errors ($s/\sqrt{n}$), confidence intervals, t-tests and many other tools.

Effect of n on variability

- **Sampling mean.** Its variance across hackers is σ^2/n: it falls by about 10 each time n grows tenfold (0.2316 at n = 10, 0.0239 at n = 100, 0.00243 at n = 1000). Estimates of μ from long trajectories are more concentrated, so the estimate becomes more reliable (consistency).
- **Sampling variance.** Its variance also decreases about like 1/n (1.185 at n = 10, 0.1031 at n = 100, 0.01027 at n = 1000). For small n it is large: at n = 2 the variance of s^2 (12.5) is about five times its mean (2.4), so a single s^2 can be far from σ^2. s^2 is also more variable than M in relative terms, because it depends on the fourth moment μ_4 of the scores.
- **Practical meaning.** Increasing the number of systems attempted by each hacker gives a sharper estimate of the hacker's average score and of the consistency of their results. The accuracy improves only like $1/\sqrt{n}$ in standard deviation: 100 times more systems are needed for 10 times more precision.